# CIS 5601: ARTIFICIAL INTELLIGENCE FINAL PROJECT

**Amitai Goldmeer**
Temple University
Philadelphia, PA
amitai.goldmeer@temple.edu

**ABSTRACT**

Knowledge Graphs are widely used to represent structured information but often assume binary truth values for entities and relationships, which limits their applicability for domains where uncertain data is widespread. This paper presents a Bayesian Knowledge Graph (BKG) framework which alters the traditional subject-predicate-object triple format with confidence values, and applies Bayesian belief updating to define relationship strength over time. Utilizing beta-distributed priors, the system locally updates edge beliefs, node reliability, and predicate-level trustworthiness as new evidence enters the system. Unlike a traditional Bayesian Network, it does not utilize conditional probability tables, enforce acyclic graphs, or compute global probability distribution, instead performing localized belief aggregation. Initial results suggest that the BKG framework offers an uncertainty-aware alternative for knowledge representation.

## 1 Introduction

Knowledge Graph systems are traditionally systems with a binary understanding of truth values. If a entity or relationship exists inside the graph structure, then it exists. If it is not within the structure, then it does not exist. Both of these are certainties. While this type of data structure has its usage in traditional knowledge spaces & in the AI space, it leaves room for improvement or change. The failure of traditional KG systems to handle uncertain data leaves much to be desired, especially as the connection and visualization of data in spaces which use uncertain data could be highly beneficial. The goal of this project was to attempt the construction of an KG system which could support uncertain data, and utilize the confidence levels of all data within the system to better estimate the certainty of that data via Bayesian statistics.

## 2 Technical Specifications

### 2.1 Knowledge Graph

Knowledge graphs traditionally not only have nodes and edges (qualifying them as graphs) but utilize specific ontologies and schema. That was not utilized in this project. This project is a 'knowledge graph' such that it visualizes uncertain knowledge in a graphical format. A visual software package 'PyVis' (built off of the graph package NetworkX and Vis.js for JS) was utilized for the visualization, and it allows a prebuilt amount of searching based on nodes and edges.

There is a larger graphical portion at work (discussed more in the Bayesian section) to re-balance the confidence values of nodes and edges. Unlike traditional Bayesian Networks which re-weight all confidence globally, this is done locally based on the edges and nodes.

For future usage, a port over to a proper knowledge graph format & software would likely allow for better utilization of the graphical format, including adding dedicated schema and ontology for entity recognition. The data processing for Bayesian networking would be identical, and using a system such as Neo4j with the Cypher language would allow for more flexibility, and take graphical packaging out of the researchers hands. It would require additional work for entity recognition, perhaps utilizing an LLM as a middleman, due to the processing of datasets ad hoc.

### 2.2 Bayesian Weighting

The principal idea was to imitate a Bayesian Network and apply it in a graphical format. By combining the way that Bayesian networks are able to make statistical determinations on forms of uncertain data with the interconnectedness of graphed knowledge, the proposed product would allow for intelligent decision making. Ideally, relationships (edges) would re-weight themselves as new data was added based on the added knowledge into the system.

The end result is correctly noted as Bayesian, but has strayed from a Bayesian Network. A traditional Bayesian Network is based on a number of items, including the following:

- Conditional Probability Tables
- Global & Multiplicative updating
- Graphically, Bayesian Networks are DAGs

Bayesian Networks computes probabilities over Bayes' Rule across an entire graph (global probabilistic inference). Comparatively, this project (Bayesian Knowledge Graph or BKG):

- Utilizes local linear belief updating
- Does not use conditional probability tables

- Allows graphical cycles
- Learns predicate priors as opposed to assuming them

It is, however, still Bayesian. The system operates at the core level of Bayesian belief updating, via the usage of beta distribution ($\alpha, \beta$) as a conjugate Bayesian prior. New observations update beliefs using the distributions, which is Bayesian-consistent. Uncertainty (the core principle on display) is represented via the beta distribution, per Bayesian principles.

## 3 Iterations

### 3.1 Iteration 1: Basic Knowledge Graph

Before attempting to integrate any amount of Bayesian weighting, I wanted to make sure that creating a graphical format was possible to any degree. My choice to use the PyVis library was a practical choice for a P.O.C. project like this, as it is relatively simple to use and integrates easily with a Python based project.

Creating a simple graph using this library is not complex, simply loading in the data from a CSV, applying the given weights as metadata for the edges, and rendering. This allowed a first look at what the data looks like before Bayesian Weighting, and was used for comparative purposes in the project final demo.

### 3.2 Iteration 2: NARS-like Bayesian Knowledge Graph

This version was one that while flawed, was highly informative. It utilized an $\alpha/\beta$ distribution, where $\alpha$ represented the count of positive evidence, and $\beta$ represented the count of negative evidence (in this case, the negative evidence against known positive evidence). Given a starting 'prior strength' (set to 2 in this iteration), $\alpha = \beta = 2$ created an initial belief of $\frac{2}{2+2} = 0.5 = 50\%$ confidence. A prior strength of 2 instead of 1 required a higher amount of evidence to shift beliefs. When a new relationship was observed, given confidence c, a weighted confidence was calculated and incremented as $\alpha = \alpha + c$, and $\beta$ was incremented to $\beta = \beta + (1 - c)$.

A concept of 'node reliability' was introduced as part of the weighted confidence calculation, each of which also had $\alpha, \beta$ values to determine how trustworthy a node was based on the quality of relationships it was attached to. This created a feedback loop of reliability, wherein consistent addition of evidence would increase the reliability of nodes and the confidence of edges.

This is referred to as 'NARS-like' due to the direct way in which data reliability and confidence is affected. In NARS, confidence & frequency are determined by repeated instances of an object and its confidence being added to the system. While this is very useful for NARS as a reasoning system, it is less helpful in this system. It would require consistent affirmation of the same knowledge to affect the confidence of uncertain data, which is not guaranteed or likely to happen, depending on circumstance of evidence. The goal was to allow all incoming data to affect other data, both positively and negatively, for belief updating based on surrounding data. This goal may have been achievable using this method in other formats, but as a knowledge graph type structure, there are rarely duplicate triples.

### 3.3 Iteration 3: Bayesian Knowledge Graph

The final result is a system which still operates in a manner similar to reasoning systems including NARS, but far more fitting for a knowledge graph system which operates over mostly unique triples. This system is inspired by NARS-style evidence accumulation, but does not implement the semantics, calculus, or inference rules of that system. Instead of working with global updating (Bayesian Networks) or reinforced belief updating (NARS), the BKG system updates confidence locally and in-line based on graphical connections as a type of evidence aggregation.

It uses beta distributions for edge-level beliefs, node-level reliability, and predicate-level priors. Nodes and predicates are initialized with $\alpha = \beta = 0.5$ to represent a prior with an uninformative prior, and edges are initialized from a predicate prior (confidence) when seen. Node reliability is calculated as such when a triple (Subject, Predicate, Object, Confidence) is ingested:

- Subject Reliability = $\alpha$ Subject / ($\alpha$ Subject + $\beta$ Subject)
- Object Reliability = $\alpha$ Object / ($\alpha$ Object + $\beta$ Object)
- Node Weight = (Subject Reliability + Object Reliability) / 2

Edges are initialized if they do not exist, and inherit the *learned statistic* of the predicate type. For example, if the edge 'worksAt' is a newly initialized type, it starts with (0.5,0.5) at a confidence of 50%. In the same example, the 3rd 'worksAt' edge might start with (3.2, 1.8) at a confidence of 65%.

From there, evidence strength is calculated by multiplying the calculated node weight with the evidence scale (in the system currently, set to 3). This takes how reliable the nodes are with an amplification factor, and determines how strong a piece of evidence should be taken at. Given an edge with confidence 'c', and evidence strength 's' it's $\alpha$ is calculated as $\alpha = \alpha + c * s$ and $\beta = \beta + (1 - c) * s$

$$(\alpha_e, \beta_e) \longleftarrow (\alpha_e + c * s, \beta_e + (1 - c) * s)$$

Equation for Bayesian weighting (belief updating) rule for edges in BKG system

Once edge belief has been updated, node reliability can be updated as well. This is done using the raw confidence, not the version scaled by evidence strength. $\alpha$ and $\beta$ are recalculated by adding confidence and 1-confidence, and reliability is recalculated by $\frac{\alpha}{\alpha+\beta}$. Low confidence edges reduce node reliability, and high confidence edges increase node reliability. Lastly, the predicate prior is updated. This is done in the same manner as the node reliability is, using raw confidence.

This design creates a positive feedback loop in which high-confidence edges increase node reliability, and high-reliability nodes amplify future evidence. Predicate-specific learning ensures that each relationship type learns and stabilizes independently, balancing a large re-weighting against individual reliability metrics. Utilizing an evidence scale/strength allows users to determine a workable scale of how much evidence should be valued with each instance based on the circumstance of usage, and the amount of evidence.

## 4 Practical Example

Using a dataset of medical diseases and symptoms, with randomly generated confidence values from 0.2 - 0.9, two graphs were generated. One using just the basic KG format which shows the confidence values of the edges as they were generated, and one which has the Bayesian weighting and node reliability of the BKG system. While this dataset is not wholly accurate, it still demonstrates the power of the Bayesian weighting and local updating. In rare cases, edge confidence was brought down compared to the original confidence value, but mostly edge confidence was brought. While this was expected to an extent, it may be over-represented due to some graph cycles, and overall the size of the dataset. There are some concerns about feedback loops over graph cycles consistently updating the confidence values too high.

## 5 Improvements

A number of potential improvements are reasonable for this system. As already noted, porting the graph over to a proper software for KG display such as Neo4j would bring varied benefits including better visualization, usage of entity, schema, and ontology recognition, and more. For the BKG system itself, looking into methods to determine proper evidence scale and initial $\alpha, \beta$ values would not be out of the question, as right now the values used are satisfactory, but there is no determined scale based on amount and type of evidence. Looking into potential cycle feedback loops and how to avoid them would be beneficial (potentially borrowing algorithms from graph traversal or PageRank in how to avoid over-increasing confidence via cycle). Additionally, updating the system to allow additional updates instead of only allowing bulk loading would be highly beneficial from a research and practical viewpoint.

## 6 Conclusion

This project explored the construction of a Bayesian Knowledge Graph that combines the flexibility of traditional knowledge graphs with Bayesian belief updating to reason over uncertain information. The system enables uncertainty-aware knowledge representation without using conditional probability tables or acyclic graphs. Unlike traditional Bayesian Networks, the BKG system per-forms local belief updates as opposed to global inference. This design makes for a well-suited tool in domains where knowledge is incomplete or consistently evolving such as medical or scientific fields. While this is not meant as a replacement for Bayesian Networks, it offers a complementary framework for uncertain knowledge aggregation. The Bayesian Knowledge Graph represents a middle ground in and underrepresented field between symbolic knowledge and probabilistic reasoning.

## References


[1] https://drops.dagstuhl.de/entities/document/10.4230/TGDK.3.1.3

[2] https://link.springer.com/article/10.1007/s11704-023-2427-z

[3] https://journals.plos.org/ploscompbiol/article?id=10.1371%2Fjournal.pcbi.1011945&

[4] https://arxiv.org/html/2405.16929v2

[5] https://jmlr.org/papers/volume18/15-631/15-631.pdf

[6] https://homes.cs.washington.edu/~pedrod/papers/mlj05.pdf

[7] https://aaai.org/papers/01135-AAAI00-204-non-axiomatic-reasoning-system-version-4-1/

[8] https://www.sciencedirect.com/book/monograph/9780080514895/probabilistic-reasoning-in-intelligent-systems

[9] https://openreview.net/forum?id=4odAwBAhQ2&referrer=[the+profile+of+Yuan+He](%2Fprofile%3Fid%3D~Yuan_He5)

[10] https://arxiv.org/abs/2410.08985

[11] https://medium.com/%40jain.sm/beyond-brittle-facts-building-a-probabilistic-knowledge-graph-046d60a9c

[12] https://github.com/allthingssecurity/probabilistic_kg

[13] *Presentation Examples* https://github.com/ahgoldmeer/Med_KG

[14] *PyVis Docs*https://pyvis.readthedocs.io/en/latest/index.html

[15] *Bayesian Network Visualization* https://www.bayesserver.com/Visualizations/ForceDirected.aspx

[16] *NAL Wiki*. https://github.com/opennars/opennars/wiki

[17] *Visualization of Uncertainty and Reasoning*. https://link.springer.com/chapter/10.1007/978-3-540-73214-3_15